EVALUATION GUIDE

RAGAS

Retrieval Augmented Generation Assessment

A complete, step-by-step guide to evaluating your RAG pipeline. From collecting data to diagnosing problems and improving scores — everything you need in one document.

Production Reference • v1.0

Contents

Visual Evaluation Flow

Step 1 — Install RAGAS

Step 2 — Collect Outputs from Your RAG Pipeline

- What to capture
- Where do questions come from?
- How many questions?

Step 3 — Pick Your Metrics

- Path A: No ground truth
- Path B: With ground truth
- Which component does each metric evaluate?

Step 4 — Choose Your Evaluator LLM

- Three rules for choosing
- Setup: OpenAI / Claude / Ollama

Step 5 — Run the Evaluation

- Full working code

Step 6 — Understand Your Scores

- Score interpretation
- Per-sample analysis

Step 7 — Diagnose and Fix Problems

- Retriever problems

Generator problems

Step 8 — Re-evaluate

Appendix A — Generating a Synthetic Test Set

Appendix B — Common Mistakes to Avoid

Appendix C — Score Reference Card

Quick Start Checklist

Visual Evaluation Flow

Follow this flow from top to bottom. Each box is one step. The detailed guide for each step follows after this chart.

STEP 1 **Install RAGAS**

```
pip install ragas datasets langchain-openai
```

↓

STEP 2 **Run Your RAG Pipeline on a Batch of Questions**

For each question, capture: **question** + **contexts** (retrieved chunks) + **answer** (generated response)

Minimum 20 questions. Use real user queries where possible.

↓

STEP 3 **Do You Have Ground Truth Answers?**

Ground truth = human-written correct answer for each question

NO GROUND TRUTH

Metrics to use:

- **Faithfulness** — catches hallucination
- **Answer Relevancy** — catches off-topic answers

These two alone catch the most common RAG failures.

HAS GROUND TRUTH

Metrics to use:

- Faithfulness + Answer Relevancy
- **+ Context Precision** — retrieval ranking
- **+ Context Recall** — retrieval completeness
- **+ Answer Correctness** — factual accuracy

↓

STEP 4 Choose Your Evaluator LLM

A *separate* LLM that judges your pipeline. Not the LLM inside your RAG.

Rule: Evaluator ≥ Generator in capability. Different model family preferred.

Default: OpenAI (just set API key). Or use Claude / Ollama.

↓

STEP 5 Run Evaluation

```
result = evaluate(dataset, metrics=[...], llm=..., embeddings=...)
```

Output: score **0** → **1** for each metric. Higher is better.

↓

STEP 6 Read Your Scores

0.0 – 0.5 Problem	0.5 – 0.8 Needs Work	0.8 – 1.0 Good

Always check per-sample scores, not just averages.

↓

STEP 7 Diagnose — Where Is the Problem?

```
Question → [ RETRIEVER ] → contexts → [ GENERATOR ] → answer
```

Retriever Problem

Low Context Precision → re-ranker, better embeddings

Low Context Recall → increase top-k, hybrid search

Generator Problem

Low Faithfulness → tighten prompt, lower temperature

Low Relevancy → improve prompt, few-shot examples

Low Correctness → stronger model, more context

↓

STEP 8 Fix → Re-evaluate → Repeat

Make changes → re-run on the **same questions** → compare scores → iterate until satisfied.

1 Install RAGAS

```
# Core packages
pip install ragas datasets

# Pick ONE based on your evaluator LLM choice (Step 4):
pip install langchain-openai        # OpenAI (default, simplest)
pip install langchain-anthropic     # Claude
pip install langchain-ollama        # Local models via Ollama (free)
```

TIP

If you're unsure, start with `pip install ragas datasets langchain-openai`. You can always add other providers later.

2 Collect Outputs from Your RAG Pipeline

RAGAS evaluates **outputs your pipeline has already produced**. You run your RAG, capture the results, and hand them to RAGAS for scoring.

What to Capture

FIELD	WHAT IT IS	TYPE	REQUIRED?
`question`	The user's query	string	Always
`answer`	Your pipeline's generated answer	string	Always
`contexts`	Retrieved text chunks	list of strings	Always
`ground_truth`	Human-written correct answer	string	Only for some metrics

How to Collect

```
# Adapt this to your pipeline
questions = [
    "What is photosynthesis?",
    "How does gravity work?",
    "What causes rain?",
]

all_questions, all_answers, all_contexts = [], [], []

for q in questions:
    # ── Replace these with YOUR pipeline ──
    retrieved_chunks = your_retriever.retrieve(q)
    generated_answer = your_llm.generate(q, retrieved_chunks)
    # ──────────────────────────────────────

    all_questions.append(q)
    all_answers.append(generated_answer)
    all_contexts.append(retrieved_chunks)  # list of strings
```

Where Do the Questions Come From?

SOURCE	HOW	BEST FOR
Write them yourself	Think of 20–50 questions a user would ask about your docs	Getting started fast
Pull from logs	Sample real questions from your app's query logs	Real-world accuracy
Generate synthetically	Use RAGAS TestsetGenerator (see Appendix A)	Bootstrapping + ground truth

How Many Questions?

GOAL	SAMPLE SIZE
Quick sanity check — "does this work at all?"	10 – 20
Comparing two pipeline configs	50 – 100
Serious benchmarking	200+

RECOMMENDATION

Start with 20 questions. You can always add more later. Getting scores on 20 real questions is infinitely better than having no evaluation at all.

3 Pick Your Metrics

The Key Question: Do You Have Ground Truth?

Ground truth = a human-written correct answer for each question. **Most people don't have this when starting out. That's completely fine.**

Path A: No Ground Truth — Start Here

These two metrics need only `question`, `answer`, and `contexts`:

METRIC	WHAT IT CHECKS	LOW SCORE MEANS
Faithfulness No GT needed	Is the answer backed by the retrieved context?	Your LLM is hallucinating — making up info not in the chunks
Answer Relevancy No GT needed	Does the answer address the question?	Your LLM is going off-topic or giving vague responses

```
from ragas.metrics import faithfulness, answer_relevancy
metrics = [faithfulness, answer_relevancy]
```

WHY THESE TWO ARE ENOUGH TO START

Faithfulness catches hallucination. Answer Relevancy catches irrelevance. Together, they cover the two most common RAG failures. You can add more metrics later.

Path B: With Ground Truth — Full Evaluation

If you have reference answers, add these for deeper retriever and generator insight:

METRIC	WHAT IT CHECKS	LOW SCORE MEANS
Context Recall Needs GT	Did retrieval find all info needed to answer?	Your retriever is missing relevant chunks
Context Precision Needs GT	Are relevant chunks ranked at the top?	Your retriever returns relevant stuff but buries it under noise
Answer Correctness Needs GT	Is the answer factually correct vs. reference?	Your LLM gives wrong answers even when context is fine

```
from ragas.metrics import (
    faithfulness, answer_relevancy,
    context_precision, context_recall,
    answer_correctness,
)
metrics = [faithfulness, answer_relevancy, context_precision,
           context_recall, answer_correctness]
```

Which Component Does Each Metric Evaluate?

```
Question → [ RETRIEVER ] → contexts → [ GENERATOR ] → answer
```

COMPONENT	METRICS	WHAT YOU LEARN
Retriever	Context Precision, Context Recall	Is the retriever fetching the right chunks?
Generator	Faithfulness, Answer Relevancy, Answer Correctness	Is the LLM producing good answers from those chunks?

This split tells you **where the problem is** so you fix the right component.

4 Choose Your Evaluator LLM

RAGAS uses an LLM to **judge** your pipeline's outputs. This is **not** the LLM in your RAG pipeline — it's a separate model used only for evaluation.

Three Rules for Choosing

RULE 1

The evaluator should be **at least as capable** as your generator. A weak model cannot reliably judge a stronger model's output.

RULE 2

Use a **different model family** than your generator. Models tend to rate their own outputs higher (self-bias). If your RAG uses GPT-4o, evaluate with Claude — and vice versa.

RULE 3

When in doubt, use **GPT-4o** or **Claude Sonnet**. These are reliable for most use cases.

Recommendation by Situation

YOUR SITUATION	RECOMMENDED EVALUATOR	WHY
Just getting started	GPT-4o-mini	Cheap, fast, good enough for directional signals
Comparing pipeline variants	GPT-4o or Claude Sonnet	Reliable, good cost/quality balance
High-stakes final eval	Claude Opus or GPT-4o	Most accurate judgments
No API budget	Llama 3 via Ollama	Free, but less reliable

Setup: Option A — OpenAI (Simplest)

```
import os
os.environ["OPENAI_API_KEY"] = "sk-..."

# That's it. RAGAS uses OpenAI by default.
# No additional LLM setup needed — skip to Step 5.
```

Setup: Option B — Claude

```
import os
os.environ["ANTHROPIC_API_KEY"] = "sk-ant-..."

from ragas.llms import LangchainLLMWrapper
from ragas.embeddings import LangchainEmbeddingsWrapper
from langchain_anthropic import ChatAnthropic
from langchain_openai import OpenAIEmbeddings

evaluator_llm = LangchainLLMWrapper(
    ChatAnthropic(model="claude-sonnet-4-20250514")
)
evaluator_embeddings = LangchainEmbeddingsWrapper(OpenAIEmbeddings())
# Note: You can use OpenAI embeddings even with Claude as evaluator.
```

Setup: Option C — Local Model via Ollama (Free)

```
# Terminal: install Ollama and pull a model
# ollama pull llama3

from ragas.llms import LangchainLLMWrapper
from ragas.embeddings import LangchainEmbeddingsWrapper
from langchain_ollama import ChatOllama, OllamaEmbeddings

evaluator_llm = LangchainLLMWrapper(ChatOllama(model="llama3"))
evaluator_embeddings = LangchainEmbeddingsWrapper(
    OllamaEmbeddings(model="llama3")
)
```

How to Verify Your Evaluator Is Good Enough

Run the same evaluation twice on the same data. If scores differ significantly between runs (e.g., Faithfulness jumps from 0.6 to 0.85), your evaluator model is too weak. Switch to a stronger model.

5 Run the Evaluation

Combine everything from the previous steps into one script. **Copy, adapt the data section, and run.**

```python
# ═══════════════════════════════════════════════════════════
# RAGAS Evaluation Script — Copy and Adapt
# ═══════════════════════════════════════════════════════════

import os
from datasets import Dataset
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy

# ── 1. SET YOUR API KEY ──
os.environ["OPENAI_API_KEY"] = "sk-..."

# ── 2. YOUR RAG PIPELINE OUTPUTS ──
# Replace this with actual outputs from your pipeline
data = {
    "question": [
        "What are the benefits of exercise?",
        "How does gravity work?",
        "What causes rain?",
    ],
    "answer": [
        "Exercise improves heart health, boosts mood, and helps manage weight.",
        "Gravity is a force that attracts objects with mass toward each other.",
        "Rain is caused by water evaporating, forming clouds, and condensing.",
    ],
    "contexts": [
        ["Regular physical activity reduces heart disease risk and aids weight control."],
        ["Gravity is a fundamental force. It causes objects with mass to attract one another."],
        ["The water cycle involves evaporation, condensation in clouds, and precipitation."],
    ],
}

# If you have ground truth, uncomment:
# data["ground_truth"] = ["reference 1", "reference 2", "reference 3"]

# ── 3. CHOOSE METRICS ──
metrics = [faithfulness, answer_relevancy]

# With ground truth, use more metrics:
# from ragas.metrics import context_precision, context_recall, answer_correctness
# metrics = [faithfulness, answer_relevancy, context_precision,
#            context_recall, answer_correctness]

# ── 4. CREATE DATASET AND RUN ──
dataset = Dataset.from_dict(data)

result = evaluate(
    dataset,
    metrics=metrics,
    # If using non-default LLM:
    # llm=evaluator_llm,
    # embeddings=evaluator_embeddings,
)
```

```
# ── 5. VIEW RESULTS ──
print(result)
# Example: {'faithfulness': 0.92, 'answer_relevancy': 0.88}
```

THAT'S IT

You now have evaluation scores for your RAG pipeline. Each metric returns a score from 0 to 1. Higher is better.

6 Understand Your Scores

Score Interpretation

0.0 – 0.5 Significant Problem	0.5 – 0.8 Needs Improvement	0.8 – 1.0 Good

METRIC	POOR (<0.5)	ACCEPTABLE (0.5-0.8)	GOOD (>0.8)
Faithfulness	Frequent hallucination	Occasional unsupported claims	Well-grounded answers
Answer Relevancy	Often off-topic	Partially addresses question	Directly answers question
Context Precision	Mostly noise in results	Relevant results mixed with noise	Relevant results on top
Context Recall	Missing critical info	Has most of what's needed	Comprehensive retrieval
Answer Correctness	Frequently wrong	Partially correct	Factually accurate

Always Check Per-Sample Scores

An overall score of 0.75 could mean "all answers are decent" or "half are perfect, half are terrible." Always look at individual samples:

```python
df = result.to_pandas()
print(df)

# Find the worst-performing samples
print("\n--- Low Faithfulness (below 0.5) ---")
print(df[df["faithfulness"] < 0.5][["question", "answer", "faithfulness"]])

print("\n--- Low Relevancy (below 0.5) ---")
print(df[df["answer_relevancy"] < 0.5][["question", "answer", "answer_relevancy"]])
```

DON'T SKIP THIS

Reading through low-scoring samples reveals patterns: certain question types, topics, or retrieval failures that averages hide.

7 Diagnose and Fix Problems

```
Question → [ RETRIEVER ] → contexts → [ GENERATOR ] → answer
```

Retriever Problems

Indicated by low Context Precision or Context Recall

Generator Problems

Indicated by low Faithfulness, Answer Relevancy, or Answer Correctness

Retriever Fixes

SYMPTOM	CAUSE	WHAT TO TRY
Low Context Recall *Missing relevant info*	Retriever isn't finding all relevant chunks	• Increase top-k (e.g., 3 → 5 → 10) • Try smaller or overlapping chunks • Add hybrid search (semantic + keyword/BM25) • Try a better embedding model
Low Context Precision *Too much noise*	Irrelevant chunks ranked too high	• Add a re-ranker (cross-encoder) • Add metadata filters (date, category, source) • Upgrade embedding model • Improve chunk boundaries

Generator Fixes

SYMPTOM	CAUSE	WHAT TO TRY
Low Faithfulness *Hallucinating*	LLM adds info not in the context	• Add to prompt: "Only use the provided context. If the answer isn't in the context, say you don't know." • Set temperature to 0 or 0.1 • Use a more instruction-following model
Low Answer Relevancy *Off-topic answers*	LLM doesn't address the question	• Be explicit: "Answer the user's question directly and concisely" • Add few-shot examples • Check if noisy context is confusing the model
Low Answer Correctness *Wrong answers*	LLM misinterprets or misses info	• Upgrade to a stronger generator model • Add chain-of-thought prompting • Provide more context chunks

8 Re-evaluate

After making changes to your pipeline:

1. **Run your updated pipeline** on the **same set of questions**
2. **Run RAGAS again** with the same metrics
3. **Compare scores** — did they improve?

```
# Before: tightened prompt, lowered temperature
# {'faithfulness': 0.65, 'answer_relevancy': 0.72}

# After:
# {'faithfulness': 0.84, 'answer_relevancy': 0.78}  ← improvement!
```

> **CRITICAL**
>
> Always compare on the **same test set**. If you change both the pipeline and the questions, you can't tell what caused the score change.

Repeat this cycle until your scores meet your target thresholds. For most applications, Faithfulness > 0.8 and Answer Relevancy > 0.7 is a strong starting point.

Appendix A: Generating a Synthetic Test Set

If you don't have questions to test with, RAGAS can generate them automatically from your documents. This also produces ground truth answers, unlocking all metrics.

```
from ragas.testset import TestsetGenerator
from ragas.testset.evolutions import simple, reasoning, multi_context
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_community.document_loaders import DirectoryLoader

# Load your documents
loader = DirectoryLoader("path/to/your/docs/")
documents = loader.load()

# Generate test set
generator = TestsetGenerator.from_langchain(
    generator_llm=ChatOpenAI(model="gpt-4o"),
    critic_llm=ChatOpenAI(model="gpt-4o"),
    embeddings=OpenAIEmbeddings(),
)

testset = generator.generate_with_langchain_docs(
    documents,
    test_size=50,
    distributions={
        simple: 0.5,          # straightforward factual questions
        reasoning: 0.25,      # requires reasoning over context
        multi_context: 0.25,  # needs info from multiple chunks
    },
)

test_df = testset.to_pandas()
print(test_df.head())
# Columns: question, ground_truth, contexts, evolution_type
```

IMPORTANT

Synthetic questions are a great starting point, but always mix in real user queries once you have them. Synthetic-only test sets may not reflect actual user behavior.

Appendix B: Common Mistakes to Avoid

MISTAKE	WHY IT'S A PROBLEM	DO THIS INSTEAD
Testing with only 5 questions	Scores are statistically meaningless	Use at least 20 questions, ideally 50+
Only looking at average scores	Hides a subset of completely broken answers	Always inspect per-sample scores and low outliers
Same LLM as judge and generator	Self-evaluation bias inflates scores	Use a different model family for evaluation
Changing test set between experiments	Can't attribute score changes to your fix	Keep the test set fixed when comparing variants
Only evaluating once	Pipeline quality drifts with new docs, model updates	Re-evaluate periodically, especially after changes
Hand-picking contexts for eval	Tests your LLM but not your retriever	Always use contexts actually returned by your retriever
Skipping Faithfulness metric	Missing the #1 RAG failure mode: hallucination	Always include Faithfulness in your metrics
Using a weak evaluator model	Inconsistent, unreliable scores	Use GPT-4o or Claude Sonnet at minimum for eval

Appendix C: Score Reference Card

METRIC	NEEDS GROUND TRUTH	EVALUATES	WHAT IT CATCHES
Faithfulness	No	Generator	Hallucination — LLM invents info not in the context
Answer Relevancy	No	Generator	Off-topic or vague answers that don't address the question
Context Precision	Yes	Retriever	Irrelevant chunks ranked too high in results
Context Recall	Yes	Retriever	Retriever fails to find all relevant information
Answer Correctness	Yes	Generator	Factually wrong answers despite having correct context

Quick Start Checklist

1. Install: `pip install ragas datasets langchain-openai`
2. Set your API key: `os.environ["OPENAI_API_KEY"] = "sk-..."`
3. Run your RAG pipeline on 20+ questions — collect `question`, `answer`, `contexts`
4. Run: `evaluate(dataset, metrics=[faithfulness, answer_relevancy])`
5. Check overall scores — are they above 0.8?
6. Inspect low-scoring samples — what's going wrong?
7. Fix the problem (prompt, retriever, chunking, model)
8. Re-run on the **same questions** — did scores improve?
9. Repeat until satisfied